



TECNOLÓGICO
NACIONAL DE MÉXICO®



ANTEPROYECTO

MATERIA:

GRAFICACIÓN

NOMBRE DEL PROYECTO:

Simulador de Deformación Estructural y Mapeo de Tensiones en Tiempo Real mediante Python y OpenGL

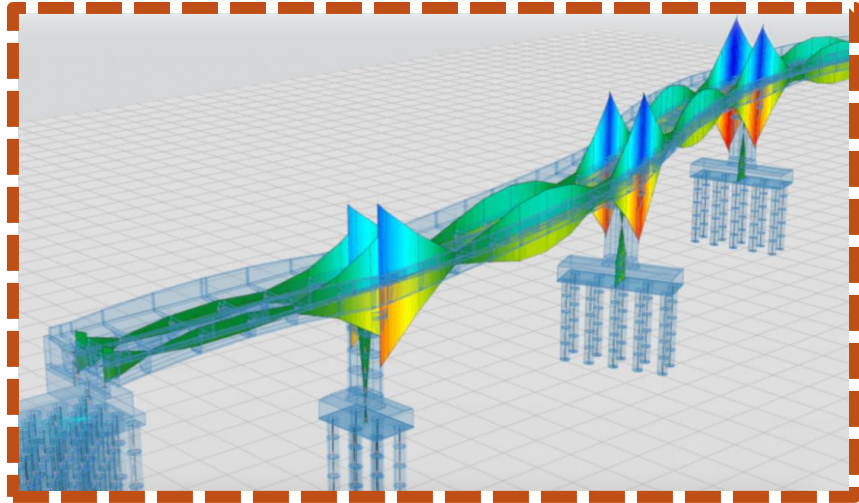
MAESTRO:

EURESTY URIBE CARLOS GERARDO

ALUMNOS:

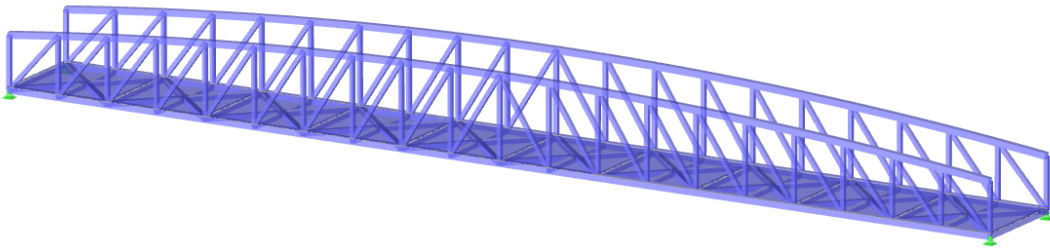
HERNÁNDEZ LÓPEZ CLARA PAULINA

SERRANO SERRANO DIEGO



OBJETIVO GENERAL

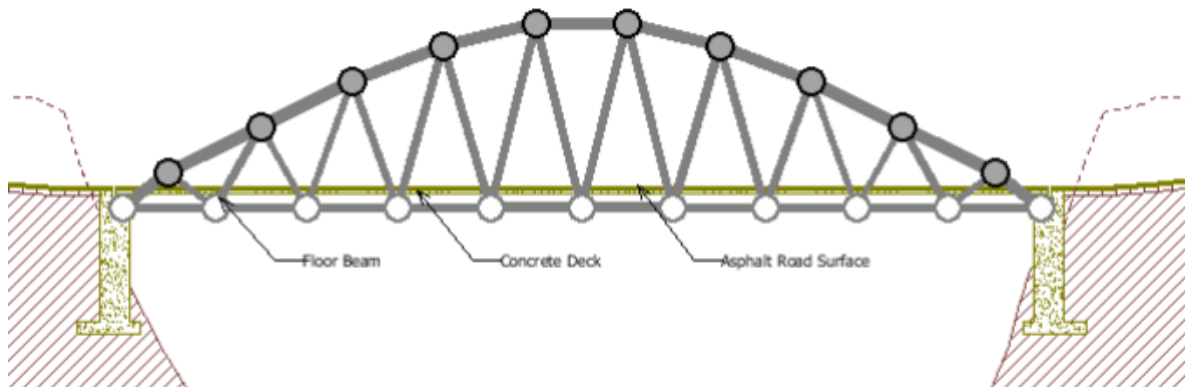
Desarrollar un simulador gráfico interactivo en 3D utilizando Python y OpenGL que permita visualizar y analizar la respuesta mecánica de un puente de armadura ante cargas dinámicas. El propósito es crear una herramienta educativa donde la computación gráfica sirva como puente para entender fenómenos físicos complejos, como la deformación elástica y la distribución de esfuerzos en tiempo real.



Objetivos Específicos

- Para alcanzar el objetivo general, necesitamos como equipo cumplir y desarrollar los siguientes puntos técnicos:
- Implementar un motor de renderizado dinámico: Configurar un entorno en OpenGL que no solo dibuje una estructura estática, sino que también permita la actualización constante de los vértices (nodos) en la GPU, permitiendo que el puente se deforme en la animación visualmente según los cálculos físicos dependiendo de que fenómeno esté sucediendo en el puente, ya sea algún automóvil grande, pequeño, a gran o poca velocidad.
- Programar la lógica de "Cargas Dinámicas": Desarrollar un algoritmo que simule el paso de un vehículo sobre la estructura, transfiriendo su peso de forma proporcional entre los nodos a medida que se desplaza por el eje de la carretera.
- Codificar un sistema de visualización de esfuerzos (Stress Mapping): Crear una función que traduzca las fuerzas internas de tensión y compresión calculadas matemáticamente en un gradiente cromático (del azul al rojo), permitiendo la interpretación visual de los puntos críticos de la estructura.

- Simular la elasticidad y oscilación: Aplicar modelos matemáticos (como la Ley de Hooke y amortiguamiento básico) para que el movimiento de la estructura no sea rígido, sino que presente vibraciones y deflexiones realistas ante el impacto de la carga.
- Establecer una interfaz interactiva básica: Permitir que el usuario modifique variables en tiempo real, como la masa del vehículo o la velocidad, y observe de inmediato cómo estas decisiones afectan la estabilidad e integridad visual del puente.



Antecedentes y Proyectos Similares

Para entender que es lo queremos obtener, necesitamos saber que herramientas existen ya parecidas al proyecto que vamos a desarrollar. El análisis de estructuras por computadora no es algo que sea tan innovador o que no se haya hecho ya, pero nuestro proyecto busca simplificar esa tecnología para entender la relación entre la física y los gráficos.

West Point Bridge Designer

Este es quizás el referente más famoso a nivel educativo. Fue diseñado por ingenieros militares para que estudiantes pudieran diseñar puentes de armadura y probar si resistían el paso de un camión.

Relación con nuestro proyecto: Al igual que este software, nosotros usaremos líneas para representar las vigas y nodos para las uniones. La diferencia es que nosotros vamos a programar desde cero el motor gráfico en OpenGL, lo que nos da control total sobre cómo se ve la deformación y cómo fluyen los colores del estrés estructural.

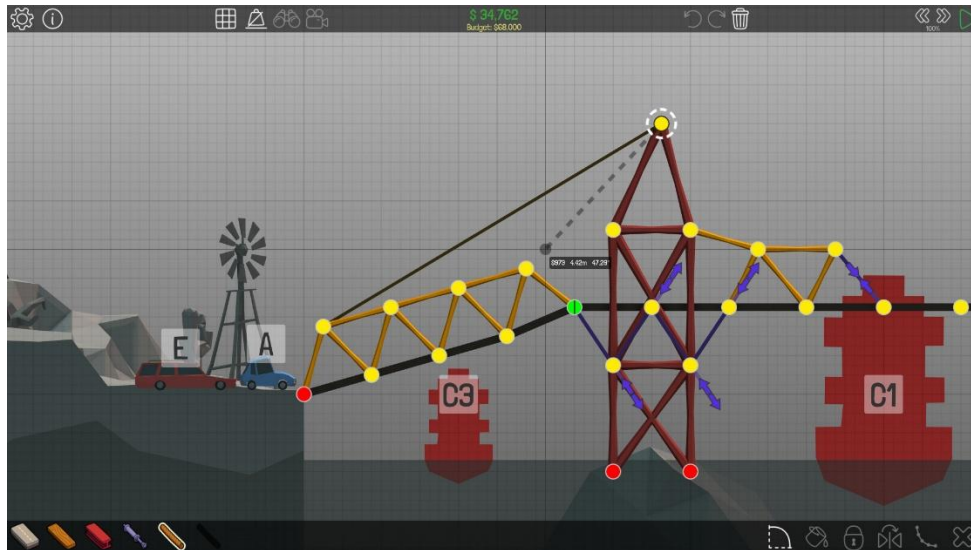


Simuladores de Física en Videojuegos (Como "Bridge Constructor" o "Poly Bridge")

Hay muchos videos en youtube de estos dos juegos, se volvieron muy famosos en su época, son juegos que consisten en ir armando un puente dependiendo de ciertos requerimientos, presupuesto y recursos limitados.

Sentido de nuestro proyecto: Esos juegos son divertidos, pero a menudo "esconden" la matemática para que el jugador solo vea el resultado. Nuestro proyecto tiene un

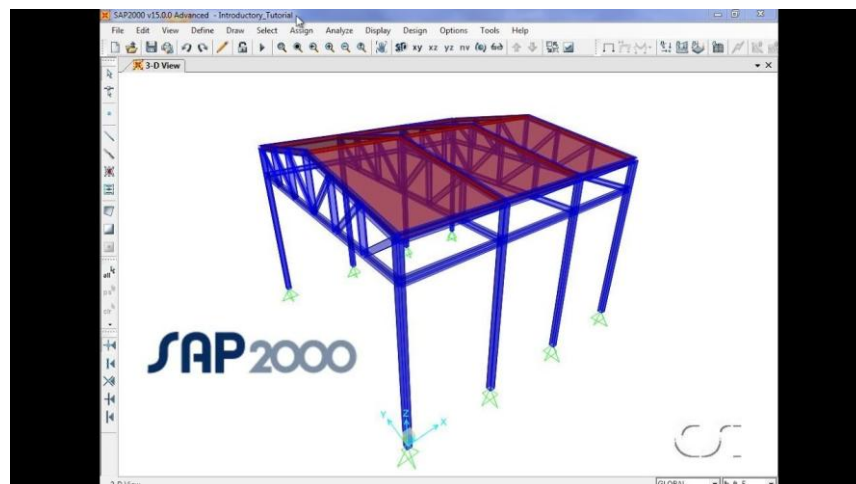
sentido académico y más analítico, mostrar en que partes el puente se muestra con más tensión.



Software de Análisis Profesional (SAP2000 y ANSYS)

Estos son los programas que usan los ingenieros civiles reales para diseñar puentes como el de San Francisco o distribuidores viales. Son programas carísimos y muy complejos.

Sentido de nuestro proyecto: No podemos decir que podemos competir con ellos, es un programa muchísimo más pequeño, pero sí podemos en algunos aspectos imitar su lógica. Estos programas usan algo llamado "Método de Elementos Finitos". Nosotros tomaremos esa idea de dividir un puente en piezas pequeñas y usaremos el poder de la tarjeta de video (OpenGL) para calcular y dibujar la respuesta de esas piezas en tiempo real, algo que incluso a los programas profesionales les costaba trabajo hace unos años.



JUSTIFICACIÓN DEL PROYECTO

En la ingeniería civil y estructural, la capacidad de predecir cómo se comportará un puente antes de ser construido es vital para la seguridad pública. Según **Hibbeler (2012)**, el análisis dinámico permite identificar fenómenos que el análisis estático ignora, como la fatiga de materiales y la resonancia. Este proyecto busca aplicar este entendimiento, aplicando teoría matemática y física al proyecto, para así entender como ciertos fenómenos influyen en la carga del puente.

Visuales

Aplicaremos ayudas visuales para entender completamente el problema, porque, aunque ver cómo se mueve el vehículo en el puente, puede dar una respuesta a como se está comportando el puente, podemos usar más herramientas para entender mejor que está pasando. Como señalan **Shreiner et al. (2013)** en la "Guía Oficial de OpenGL", la representación de datos mediante el color (mapeo de calor) permite al cerebro humano procesar información mucho más rápido que leyendo tablas de números. Al implementar un gradiente de color que reacciona a la carga, el proyecto justifica su existencia como una herramienta de diagnóstico visual en ciertos puntos de la estructura del puente.

Dificultad del proyecto

Desde el punto de vista en la materia de Graficación, este proyecto justifica su complejidad al no aplicar sólo cargas estáticas, sino dinámicas.

Aprender a animar los gráficos: Normalmente, en clase aprendemos a dibujar un objeto y dejarlo ahí. Aquí el reto es que el puente tiene que "moverse" según la física. Esto nos obliga a entender cómo actualizar los puntos (vértices) en la memoria de la tarjeta de video constantemente sin que el programa se trabe o se cierre (**Shreiner et al., 2013**).

Hacer que el código sea eficiente: Como tenemos que calcular la física y al mismo tiempo dibujar, tenemos que aprender a organizar nuestro código en un "ciclo" (loop). Si no lo hacemos bien, la simulación se verá lenta. Es un ejercicio de optimización práctica que es fundamental en cualquier software moderno (**Nystrom, 2014**).

Convertir números en colores: Este es quizás el reto más interesante de graficación. Tenemos que crear una lógica donde, si una viga recibe mucha fuerza, el programa

automáticamente cambie su color. Esto nos enseña cómo usar OpenGL para mostrar información importante y no solo para que se vea bonito.

Al utilizar Python, un lenguaje accesible pero potente, y OpenGL, el estándar de la industria, el proyecto demuestra que es posible crear simulaciones de alta fidelidad sin necesidad de software propietario costoso como SAP2000 o ANSYS. Esto nos obliga a exigirnos un poco más, en lugar de ser un simple usuario de software preexistente. Como el profesor nos mencionó, buscamos algo que nos deje y que no sea algo que se programe de la noche a la mañana, tiene cierta base, y verdaderamente nos llamó la atención analizar las cargas, y aplicar conocimientos de otras materias como física e incluso ecuaciones diferenciales en un proyecto de graficación

Planteamiento del Problema

Para que el proyecto no sea solo un puente al azar, vamos a plantear un problema real que queremos resolver con nuestra herramienta:

Imaginamos que tenemos un puente tipo armadura y queremos saber si es seguro que pase por ahí un camión de carga moderno, que es mucho más pesado que los de antes.

Actualmente, es difícil visualizar dónde sufre más un puente cuando el peso se está moviendo. Los cálculos en papel solo nos dicen el punto máximo de estrés en un lugar fijo, pero no nos muestran el "pandeo" o la vibración que ocurre mientras el vehículo avanza.

Utilizaremos el simulador para recrear este escenario. Queremos responder visualmente:

- ¿En qué parte exacta del trayecto el puente sufre más presión?
- ¿Qué vigas están en riesgo de romperse si el camión va muy rápido?
- ¿Cómo cambia la deformación si el material del puente es más flexible (madera) o más rígido (acero)?

MARCO TEORICO

Marco Matemático y Físico del Proyecto

Para que el simulador funcione, debemos considerar tres pilares: la geometría, la elasticidad (fuerzas internas) y la dinámica (movimiento).

Modelado de la Estructura (Nodos y Elementos)

El puente no se ve como una unidad sólida, sino como un Grafo Físico.

- Nodos (N): Puntos en el espacio con coordenadas. En física, cada nodo es una "partícula" con una masa asignada.
- Elementos (E): Las vigas que conectan los nodos. Matemáticamente se tratan como vectores que tienen una longitud inicial

Ley de Hooke

Es una de las partes más importantes de la parte física. Cada viga se comporta como un resorte ultra-rígido. Si el peso del coche mueve un nodo, la viga se estira o se encoge, generando una fuerza que intenta devolver el nodo a su lugar.

La ecuación que usaremos para cada viga es:

$$F = k \cdot (L_{actual} - L_0)$$

Donde k es la rigidez, definida por:

$$k = \frac{E \cdot A}{L_0}$$

- E : Módulo de Young (Elasticidad del material: Acero ≈ 200 GPa).
- A : Área transversal de la viga.
- **Referencia: Beer et al. (2017)** explican cómo esta relación define la deformación elástica en materiales reales.

5.3 Segunda Ley de Newton

El coche no es solo una imagen; es una **fuerza gravitatoria itinerante** ($W = m \cdot g$).

A medida que el coche avanza sobre el eje x , su peso se distribuye entre los dos nodos más cercanos mediante una interpolación lineal:

- Si el coche está justo sobre el nodo A, el 100% del peso va a A.
- Si está a la mitad entre A y B, 50% va a cada uno.

Para calcular cómo se mueve el puente en cada cuadro (frame), usamos la **Segunda Ley de Newton** ($F = m \cdot a$) expresada para sistemas dinámicos:

$$M\ddot{u} + C\dot{u} + Ku = F(t)$$

- **$M\ddot{u}$** : Inercia del nodo.
- **$C\dot{u}$** : Amortiguamiento (Damping), para que el puente deje de vibrar eventualmente y no sea un "resorte eterno" (**Chopra, 2014**).
- **Ku** : La resistencia de las vigas conectadas.
- **$F(t)$** : La fuerza del coche cambiando de posición en el tiempo.

5.4 Integración Numérica (El motor del tiempo)

Como OpenGL dibuja muchas veces por segundo, no podemos resolver la ecuación de arriba de forma exacta (analítica). Usaremos un método de **Integración Numérica** (posiblemente **Euler** o **Verlet**).

En cada paso de tiempo (dt):

1. Calculamos todas las fuerzas que actúan sobre un nodo.
2. Obtenemos la aceleración: $a = \frac{F_{total}}{m}$.
3. Actualizamos la velocidad: $v_{nueva} = v_{vieja} + a \cdot dt$.
4. Actualizamos la posición: $pos_{nueva} = pos_{vieja} + v_{nueva} \cdot dt$.

5.5 Cálculo de Esfuerzo para Visualización (Mapeo de Color)

Para que el puente cambie de color, calcularemos el **Esfuerzo Normal** (σ) en cada viga:

$$\sigma = \frac{F}{A}$$

- Si $\sigma > 0$: La viga está en **Tensión** (se estira) → Color **Azul**.
- Si $\sigma < 0$: La viga está en **Compresión** (se aplasta) → Color **Rojo**.

- El brillo del color dependerá de qué tan cerca esté el valor de σ del límite de ruptura del material.

CRONOGRAMA

Plantear el modelo

En OpenGL, no dibujas un puente; dibujas **puntos y líneas**. Tu primer reto es crear una estructura de datos que Python entienda y que OpenGL pueda leer rápidamente.

- **Clase Nodo:** No solo debe tener posición (x,y) , sino también propiedades físicas como masa, velocidad, aceleración y un booleano `es_fijo` (para los apoyos del puente).
- **Clase Viga (o Member):** Debe saber qué dos nodos conecta y tener propiedades de material como el **Módulo de Young (E)** y el **Área (A)**.

Debemos de definir que tipo de puente vamos a implementar. Un puente estándar, como el **Puente Warren** o el **Pratt**, para así mismo, definir completamente la estructura y el modelado del mismo.

Definir dimensiones, pesos tomando en cuenta referencias de puentes reales, y así modelar un puente realista.

Semana	Fase / Actividad	Entregable Esperado
1	Configuración y Ventana: Inicialización de PyCharm, creación de la ventana con PyOpenGL y manejo de eventos básicos.	Ventana activa (Black Screen).
2	Estructura de Datos: Creación de clases Nodo y Viga. Definición de listas de coordenadas y conexiones.	Grafo lógico en consola.
3	Renderizado Estático: Dibujo de la armadura (Truss) usando GL_LINES. Implementación de matriz de proyección (coordenadas en metros).	Esqueleto del puente en pantalla.

Semana	Fase / Actividad	Entregable Esperado
4	Interacción Básica: Implementación de cámara (zoom/pan) y selección de materiales (Acero/Madera/Concreto).	Primer Avance: Prototipo Visual.
5	Modelo de Elasticidad: Programación de la Ley de Hooke aplicada a cada viga basada en su módulo de Young.	Cálculo de fuerzas internas.
6	Integración Numérica: Implementación del algoritmo de Verlet para el movimiento de los nodos.	Puente con gravedad (caída).
7	Carga Dinámica: Programación de la masa móvil (vehículo) que recorre la vía del puente.	Vehículo cruzando el puente.
8	Amortiguamiento: Ajuste del <i>Damping</i> y estabilidad numérica para evitar oscilaciones irreales.	Segundo Avance: Simulación Física.
9	Mapeo Cromático: Implementación del gradiente de color (Azul-Tensión, Rojo-Compresión) según el esfuerzo.	Puente que cambia de color.
10	HUD y Telemetría: Visualización de datos en pantalla (estrés máximo, velocidad, peso actual).	Interfaz de usuario (UI).
11	Optimización y Shaders: Mejora del rendimiento y aplicación de Shaders básicos para mejorar la estética visual.	Versión de alta fidelidad.

Semana	Fase / Actividad	Entregable Esperado
12	Pruebas de Fallo y Cierre: Simulación de colapso estructural y redacción de la memoria técnica final.	Entrega Final y Exposición.

Conclusión

Para cerrar, este proyecto es la forma más directa de entender cómo funciona la Graficación por Computadora en el mundo real. Muchas veces en clase vemos la teoría de cómo dibujar puntos o líneas, pero aquí estamos llevando eso a otro nivel: estamos usando esos puntos para simular la realidad.

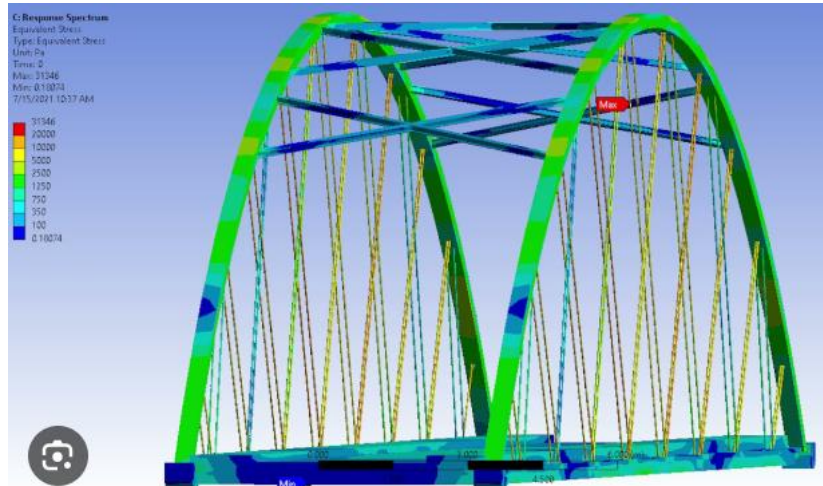
La combinación de herramientas que elegimos tiene un porqué muy claro para la materia.

El papel de OpenGL: Es el motor que nos permite hablarle directamente a la tarjeta de video. En lugar de solo hacer un dibujo que se quede "congelado", OpenGL nos da la potencia para que el puente se mueva, se doble y cambie de color 60 veces por segundo. Aprender a controlar esto es lo que separa a un programador básico de uno que sabe manejar gráficos de alto rendimiento.

El papel de Python y PyCharm: Python es genial porque nos permite escribir la lógica de la física de forma clara, sin que el código se vuelva un laberinto. Por su parte, trabajar en un entorno como PyCharm nos ayuda a que el proyecto esté organizado en carpetas y archivos limpios, algo vital porque en un simulador de 12 semanas, si el código es un desorden, el proyecto fracasa.

Al final de este semestre, no solo habremos entregado un código; habremos creado una herramienta que toma números aburridos y los convierte en colores que nos dicen si un puente es seguro o no.

Este proyecto nos enseña que la materia de graficación no se trata de hacer cosas "bonitas", sino de saber usar la computadora para representar datos de forma inteligente. Es pasar de ser alguien que usa programas hechos por otros, a ser nosotros quienes crean la tecnología para visualizar el mundo físico.



Referencias Bibliográficas

- **Beer, F. P., Johnston, E. R., & DeWolf, J. T. (2017).** *Mecánica de Materiales*. McGraw-Hill. (Base para entender la resistencia de las vigas y la Ley de Hooke).
- **Chopra, A. K. (2014).** *Dinámica de Estructuras*. Pearson Educación. (Referencia para calcular cómo vibran los puentes con cargas en movimiento).
- **Hibbeler, R. C. (2012).** *Análisis Estructural*. Pearson Educación. (Manual para el diseño de armaduras y cálculo de nodos).
- **Nystrom, R. (2014).** *Game Programming Patterns*. Genever Benning. (Guía para organizar el código en PyCharm y que la simulación sea fluida).
- **Shreiner, D., Sellers, G., Kessenich, J., & Licea-Kane, B. (2013).** *OpenGL Programming Guide: The Official Guide to Learning OpenGL*. Addison-Wesley. (La biblia para aprender a mover vértices y usar colores en la tarjeta de video).
- **Witkin, A., & Baraff, D. (2001).** *Physically Based Modeling*. SIGGRAPH Course Notes. (Explica cómo programar la física de partículas y resortes para animaciones).